GATE
CS & DA

Artificial Intelligence

Informed Search

Lecture No. 6

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

Topic AO*

Topic PDA*

Topic

Topic

Topic

Topics to be Covered

Topic IDA* ⇒ discussion

Topic BFHS

Topic Hillclimbing

Topic

Topic

The struggle you're in today is developing the strength you need for tomorrow. Don't give up.

– Robert Tew

Q. G is goal node

G expanded ?? → No need.

IDA →
- iteration is on depth ✗
- fscore
- Pruned nodes

IDA → * A* Jaisi optimal ✓

IDA → Less space Complexity O(bxd)

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The Iterative Deepening A* (IDA*) algorithm is an extension of the A* search algorithm designed to use less memory, making it more suitable for large search spaces where A* would require too much memory to store all nodes in the open and closed lists.
- IDA* combines the space efficiency of Iterative Deepening Depth-First Search (IDDFS) with the optimality of A*. Instead of maintaining a priority queue like A*, IDA* performs a series of depth-limited searches, where the depth limit is determined by a threshold that is increased iteratively. Each iteration explores paths that do not exceed the current threshold, and the threshold is based on the f-cost (sum of g-cost and h-cost).

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The evaluation function in IDA* looks like this:

$$f(n) = g(n) + h(n)$$
$$f(n) = Actual\ cost + Estimated\ cost$$

- Where h is admissible.
- here,
- f(n) = Total cost evaluation function.
- g(n) = The actual cost from the initial node to the current node.
- h(n) = Heuristic estimated cost from the current node to the goal state. it is based on the approximation according to the problem characteristics.
- Threshold: In each iteration, IDA* sets a threshold based on the f-cost. It only explores nodes where f(n)≤f(n)≤ current threshold.

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- Algorithm Steps
- Initial Threshold:
- Set the initial threshold to f(start)=g(start)+h(start), where g(start)=0.
- Depth-Limited Search:
- Perform a depth-first search (DFS) where nodes are only expanded if their f-cost does not exceed the current threshold.
- Update Threshold:
- If the goal node is not found in the current iteration, set the threshold to the minimum f-cost of any node that exceeded the current threshold in the previous iteration.
- Repeat: Repeat the DFS with the new threshold until the goal node is found or no nodes remain to explore

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Step 1: Initialization
- Set the root node as the current node, and find the f-score.
- Sep 2: Set threshold
- Set the cost limit as a threshold for a node i.e the maximum f-score allowed for that node for further explorations.
- Step 3: Node Expansion
- Expand the current node to its children and find f-scores.
- Step 4: Pruning
- If for any node the f-score > threshold, prune that node because it's considered too expensive for that node. and store it in the visited node list.
- Step 5: Return Path
- If the Goal node is found then return the path from the start node Goal node.

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Step 6: Update the Threshold
- If the Goal node is not found then repeat from step 2 by changing the threshold with the minimum pruned value from the visited node list. And Continue it until you reach the goal node.

Steps ⇒

1) It 1 ⇒ f_{th} = heuristic value of start node

2) Expand neighbour $f \leq f_{th}$, keep expanding until the condition $f \leq f_{th}$ is valid for nodes.

3) the nodes which have $f > f_{th}$ ⇒ these are pruned nodes.

Now if goal node not reached then start Ite 2, f_{th} ⇒ min f score of Pruned nodes

$A^* \Rightarrow TC\ O(b^d)$ | $IDA^* = TC\ O(b^d)$

$SC\ O(b^d)$ | $SC\ O(b \times d)$

IDDFS.

IDA* vs A*

- 1. Algorithm Type:
- A*: A* is a best-first search algorithm that explores nodes in the order of their estimated total path cost f(n)=g(n)+h(n), where g(n) is the cost to reach the node and h(n) is the heuristic estimate of the cost from the node to the goal.
- IDA*: IDA* is an iterative deepening version of A*, which uses depth-first search but applies a threshold based on f(n) to limit the depth of the search in each iteration. It increases the threshold progressively until the goal is found.

IDA* vs A*

- 2. Memory Usage:
- A*: Requires significant memory, as it stores all the nodes in the open and closed lists. In the worst case, this can lead to memory consumption growing exponentially with the depth of the search space.
- IDA*: Requires much less memory, similar to depth-first search. It only needs to store the current path and a small amount of additional information, making it much more memory-efficient.

IDA* vs A*

- 3. Time Complexity:
- A*: The time complexity is $O(b^d)$, where bb is the branching factor and dd is the depth of the optimal solution. It explores all possible paths but typically does so faster than uninformed search algorithms.
- IDA*: The time complexity is also $O(b^d)$, but due to its iterative nature, it may revisit nodes multiple times, making it potentially slower in practice compared to A*.

IDA* vs A*

- 4. Space Complexity:

- A*: Space complexity is $O(b^d)$, as it needs to store all explored nodes. This can be a significant limitation for large problems.
- IDA*: Space complexity is $O(b \times d)$, as it only stores the path being explored and the next threshold. This makes IDA* feasible for much larger problems than A*.

- Which is more Complex ⇒ IDA*

IDDFS

* max No of iteration ⇒ 3

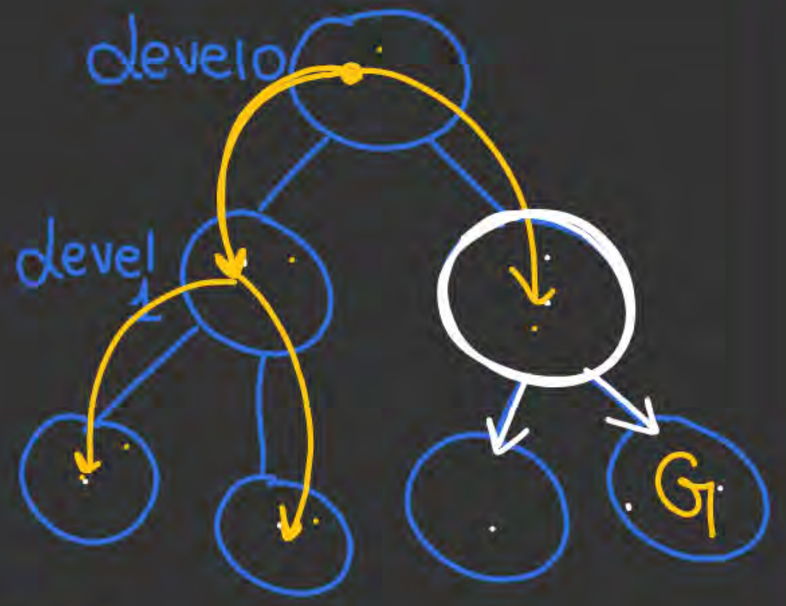
level 2

IDA*

* max No of iteration

⇒ No of nodes - 5

b=2



b: branching factor, d: depth.

In IDDFS the start node will be expanded

$O(d)$ No of times

and in IDA* $O(b^d)$ No of times

IDA* vs A*

- A* is faster in practice and is preferred when memory is sufficient.
- IDA* is more memory-efficient and suitable for large problems but may be slower due to its iterative nature.

THANK - YOU